Typefun

好的 Leo，根据我对 type.fun 的深度分析、截图UI拆解、开源生态调研，我来给你一份**详尽到骨子里**的技术方案。作为 Go 程序员，你肯定喜欢架构清晰、边界分明的东西，这份方案就按这个调性来。

🏗️ 古诗词打字练习 Web 应用 — 全栈 Node.js 技术方案

一、产品分析与功能拆解

1.1 竞品 Type.fun 核心功能分析

从你提供的三张截图来看，Type.fun 的古诗词打字模块包含两个核心页面：

📋 **课程选择页（截图一）：**

- 网格卡片布局（Grid Layout），每张卡片对应一首唐诗
- 星级评分系统（1-5星，绿色星标）表示练习成绩
- 课程解锁机制（锁图标🔒 = 未解锁）
- 多种练习模式图标（书本📖、手柄🎮、弓箭🏹等暗示不同玩法）
- 底部进度条：`已完成 / 诗词歌赋 /《唐诗三百首》3/339`
- 暗色古典中国风 UI，配合星空背景和古建筑装饰元素

⌨️ **打字练习页（截图二、三）：**

- **拼音-汉字双行对齐排版**——这是灵魂所在
- 每个汉字上方严格对齐其拼音（带声调符号 āáǎà）
- 颜色反馈机制：✅绿色 = 已正确输入，❌红色 = 输入错误，⬜灰色 = 待输入
- 当前光标位置高亮（粉红下划线指示器）
- 行尾 `↵` 符号提示按回车换行
- 顶部工具栏：重打、跟读开关、指法提示开关、拼音显示开关
- 底部统计栏：准确率 100%、按键速度 130键/分钟、打字速度 37字/分钟、进度 6%

1.2 功能模块定义

模块	核心功能	优先级
课程浏览	诗歌分类列表、搜索、筛选	P0
打字引擎	拼音逐字匹配、实时校验、颜色反馈	P0
统计系统	WPM/准确率/按键速度/进度	P0
拼音渲染	汉字-拼音对齐排版	P0
用户系统	注册/登录/进度保存	P1
星级评分	根据准确率/速度打星	P1
课程解锁	闯关解锁机制	P1
跟读朗读	TTS 语音朗读古诗	P2
指法提示	虚拟键盘指法指引	P2
排行榜	速度/准确率排名	P3

二、技术架构设计

2.1 整体架构：纯 Node.js 全栈方案

既然你要"偏向纯 Node.js"，那就别搞 Go + Node 混合了，咱直接全家桶：

代码块

```
┌──────────────────────────────────────────────────────────────┐
│                        Client (Browser)                      │
│  ┌──────────────────────────────────────────────────────────┐ │
│  │              Next.js 15 (App Router)                     │ │
│  │  ┌──────────────┐ ┌──────────────┐ ┌──────────────────┐  │ │
│  │  │ 课程浏览     │ │ 打字引擎     │ │ 统计/排行榜      │  │ │
│  │  │  页面        │ │  页面        │ │  页面            │  │ │
│  │  └──────────────┘ └──────────────┘ └──────────────────┘  │ │
│  │  ┌──────────────────────────────────────────────────┐    │ │
│  │  │      Core Typing Engine (Client-side)            │    │ │
│  │  │   • KeystrokeTracker                             │    │ │
│  │  │   • PinyinMatcher                                │    │ │
│  │  │   • RealtimeStatsCalculator                      │    │ │
```

```
14  │  │  • VisualFeedbackController              │    │ │
15  │  │  └──────────────────────────────────────────────┘    │ │
16  │  └──────────────────────────────────────────────────────┘  │
17  └────────────────────────────────────────────────────────────┘
18                          │ HTTP / WebSocket
19  ┌──────────────────────────▼──────────────────────────────┐
20  │              Next.js API Routes / Server Actions      │
21  │  ┌──────────┐  ┌──────────┐  ┌──────────┐             │
22  │  │ Auth API │  │ Poem API │  │ Stats API│             │
23  │  └────┬─────┘  └────┬─────┘  └────┬─────┘             │
24  │       │             │             │                    │
25  │  ┌────▼─────────────▼─────────────▼────┐               │
26  │  │         Prisma ORM Layer            │               │
27  │  └──────────────┬──────────────────────┘               │
28  └─────────────────┼──────────────────────────────────────┘
29                    │
30  ┌─────────────────▼──────────────────────────────────────┐
31  │  ┌──────────┐   ┌──────────┐   ┌──────────────────┐    │
32  │  │PostgreSQL│   │  Redis   │   │ chinese-poetry│    │
33  │  │ (主存储) │   │ (缓存/   │   │ (JSON 数据)  │    │
34  │  │          │   │ 排行榜) │   │                  │    │
35  │  └──────────┘   └──────────┘   └──────────────────┘    │
36  └────────────────────────────────────────────────────────┘
```

2.2 技术栈选型（100% Node.js 生态）

层级	技术选型	理由
框架	Next.js 15 (App Router)	全栈一体、SSR/SSG/CSR 灵活切换、Server Actions 省写 API
运行时	Node.js 22 LTS	最新 LTS，性能好、TypeScript 原生支持增强
语言	TypeScript 5.x	类型安全——你写 Go 的人绝对认同强类型的好处
样式	Tailwind CSS v4	原子化 CSS、暗色主题方便、响应式轻松
ORM	Prisma 6	TypeScript 生态最好的 ORM，schema-first 思路类似 Go 的 sqlc
数据库	PostgreSQL 17	可靠、JSON 支持好、全文搜索内置
缓存	Redis (ioredis)	排行榜 Sorted Set、会话缓存、实时数据
认证	NextAuth.js v5	开箱即用、支持 OAuth/邮箱/手机号
拼音库	pinyin-pro	最全的 JS 汉字转拼音库，多音字、声调、声母韵母一应俱全
诗词数据	chinese-poetry (GitHub)	5.5万首唐诗 JSON、开源、结构化
部署	Vercel / Docker + PM2	Next.js 原生 Vercel 支持，或 Docker 自建
包管理	pnpm	速度快、磁盘省——你肯定讨厌 node_modules 肿大

三、核心数据设计

3.1 诗词数据源：chinese-poetry

这个开源项目是**最全的中华古诗词数据库**，49K Star，包含 5.5万首唐诗、26万首宋诗、2.1万首宋词。数据全部是标准 JSON 格式。GitHub

唐诗数据 JSON schema 示例：

代码块

```
{
  "title": "感遇·江南有丹橘",
  "author": "张九龄",
  "paragraphs": [
    "江南有丹橘，经冬犹绿林。",
    "岂伊地气暖，自有岁寒心。",
    "可以荐嘉客，奈何阻重深。",
    "运命惟所遇，循环不可寻。",
    "徒言树桃李，此木岂无阴。"
  ],
  "tags": ["唐诗三百首", "咏物"],
  "id": "xxx"
}
```

3.2 数据库 Schema（Prisma）

代码块

```
// schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ===== 用户模块 =====
model User {
  id            String    @id @default(cuid())
  email         String?   @unique
  name          String?
  avatar        String?
```

```
  password      String?   // bcrypt hashed
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt

  sessions      Session[]
  practices     PracticeRecord[]
  courseProgress CourseProgress[]
}

model Session {
  id        String   @id @default(cuid())
  userId    String
  token     String   @unique
  expiresAt DateTime
  user      User     @relation(fields: [userId], references: [id])
}

// ===== 诗词模块 =====
model Poem {
  id           String   @id @default(cuid())
  title        String
  author       String
  dynasty      String   @default("唐")
  content      String   // 原文全文
  paragraphs   Json     // ["第一句", "第二句", ...]
  pinyinData   Json     // 预计算的拼音数据（见下方结构）
  tags         String[] // ["唐诗三百首", "咏物"]
  difficulty   Int      @default(1) // 1-5 难度等级
  charCount    Int      // 总字数（不含标点）
  sortOrder    Int      @default(0)

  category     Category @relation(fields: [categoryId], references: [id])
  categoryId   String

  practices    PracticeRecord[]
  progresses   CourseProgress[]

  @@index([categoryId, sortOrder])
  @@index([author])
}

model Category {
  id          String @id @default(cuid())
  name        String // "唐诗三百首", "宋词精选"
  slug        String @unique // "tang-poetry"
  description String?
  totalCount  Int    @default(0)
```

```
  poems       Poem[]
}

// ===== 练习记录模块 =====
model PracticeRecord {
  id            String   @id @default(cuid())
  userId        String
  poemId        String

  // 核心统计
  accuracy      Float    // 准确率 0-100
  wpm           Float    // 打字速度 (字/分钟)
  kpm           Float    // 按键速度 (键/分钟)
  correctWpm    Float    // 正确打字速度
  duration      Int      // 练习时长(秒)
  charCount     Int      // 输入总字数
  errorCount    Int      // 错误次数

  // 星级评分 (1-5)
  stars         Int      @default(0)

  // 详细击键数据 (可选，用于分析)
  keystrokeLog  Json?    // [{char, expected, timestamp, correct}, ...]

  completedAt   DateTime @default(now())

  user          User     @relation(fields: [userId], references: [id])
  poem          Poem     @relation(fields: [poemId], references: [id])

  @@index([userId, poemId])
  @@index([userId, completedAt])
}

// ===== 课程进度模块 =====
model CourseProgress {
  id            String   @id @default(cuid())
  userId        String
  poemId        String

  unlocked      Boolean  @default(false)
  bestStars     Int      @default(0) // 最高星级
  bestAccuracy  Float    @default(0) // 最高准确率
  bestWpm       Float    @default(0) // 最高速度
  attempts      Int      @default(0) // 尝试次数
  lastPosition  Int      @default(0) // 上次打字位置（断点续打）

  updatedAt     DateTime @updatedAt
```

```
  user          User     @relation(fields: [userId], references: [id])
  poem          Poem     @relation(fields: [poemId], references: [id])

  @@unique([userId, poemId])
}
```

3.3 拼音数据预处理结构

这是**整个项目最核心的数据结构**——拼音和汉字的逐字对应关系：

代码块

```
// types/pinyin.ts

interface PinyinChar {
  char: string;        // 原始字符: "江"
  pinyin: string;      // 带声调拼音: "jiāng"
  pinyinRaw: string;   // 无声调拼音(用于匹配输入): "jiang"
  tone: number;        // 声调: 1
  initial: string;     // 声母: "j"
  final: string;       // 韵母: "iang"
  isChar: boolean;     // 是否为汉字(true) / 标点(false)
  isPunctuation: boolean; // 是否标点
  position: number;    // 全局位置索引
}

interface PinyinLine {
  lineIndex: number;
  chars: PinyinChar[];
  rawText: string;     // "江南有丹橘，经"
}

interface PoemPinyinData {
  poemId: string;
  lines: PinyinLine[];
  totalChars: number;        // 汉字总数（不含标点）
  totalInputChars: number;   // 需要输入的总字符数（拼音字母 + 标点 + 回车）
}
```

3.4 数据预处理脚本

利用 `pinyin-pro` 库在导入时就预计算好所有拼音数据（pinyin-pro 官方文档）：

代码块

```typescript
// scripts/importPoems.ts
import { pinyin } from 'pinyin-pro';
import { PrismaClient } from '@prisma/client';
import poemData from './tang-poetry-300.json';

const prisma = new PrismaClient();

function generatePinyinData(text: string): PinyinChar[] {
  const result: PinyinChar[] = [];
  let position = 0;

  for (const char of text) {
    if (/[\u4e00-\u9fa5]/.test(char)) {
      // 汉字：获取拼音
      const py = pinyin(char, { toneType: 'symbol' });     // "jiāng"
      const pyRaw = pinyin(char, { toneType: 'none' });     // "jiang"
      const tone = pinyin(char, { toneType: 'num' });        // "jiang1"
      const initial = pinyin(char, { pattern: 'initial' });  // "j"
      const final_ = pinyin(char, { pattern: 'final' });     // "iang"

      result.push({
        char,
        pinyin: py,
        pinyinRaw: pyRaw.toLowerCase(),
        tone: parseInt(tone.slice(-1)) || 0,
        initial,
        final: final_,
        isChar: true,
        isPunctuation: false,
        position: position++,
      });
    } else if (/[，。！？、；：""''《》（）]/.test(char)) {
      // 中文标点
      result.push({
        char,
        pinyin: char,
        pinyinRaw: char,
        tone: 0,
        initial: '',
        final: '',
        isChar: false,
        isPunctuation: true,
        position: position++,
      });
    }
  }
  return result;
```

```
48 }
49
50 async function importPoems() {
51   for (const poem of poemData) {
52     const fullText = poem.paragraphs.join('\n');
53     const pinyinData = {
54       lines: poem.paragraphs.map((para, i) => ({
55         lineIndex: i,
56         chars: generatePinyinData(para),
57         rawText: para,
58       })),
59       totalChars: fullText.replace(/[^\u4e00-\u9fa5]/g, '').length,
60     };
61
62     await prisma.poem.create({
63       data: {
64         title: poem.title,
65         author: poem.author,
66         dynasty: '唐',
67         content: fullText,
68         paragraphs: poem.paragraphs,
69         pinyinData: pinyinData,
70         charCount: pinyinData.totalChars,
71         tags: poem.tags || [],
72         categoryId: tangPoetryCategory.id,
73       },
74     });
75   }
76 }
```

四、核心打字引擎设计（前端核心）

这是**整个应用最复杂、最核心**的部分。打字引擎完全运行在客户端，不依赖服务器交互（离线可用）。

4.1 引擎架构

代码块

```
1  ┌──────────────────────────────────────────────────┐
2  │          TypingEngine（主控制器）                  │
3  │  ┌──────────────┐  ┌──────────────────────────┐  │
4  │  │ KeyInput     │  │ PinyinStateMachine       │  │
5  │  │ Handler      │→ │ （拼音匹配状态机）          │  │
```

```
6   │  └──────────────┘  └──────────────────────┘  │
7   │                        │              │
8   │  ┌──────────────┐  ┌──────────▼──────────┐  │
9   │  │ Stats        │ │  CursorController    │ │
10  │  │ Tracker      │←│  (光标推进控制)        │ │
11  │  └──────────────┘  └──────────────────────┘  │
12  │                        │              │
13  │  ┌──────────────────────────▼──────────┐  │
14  │  │  VisualFeedback (UI状态更新)          │ │
15  │  └──────────────────────────────────────┘  │
16  └──────────────────────────────────────────────┘
```

4.2 拼音匹配状态机

这是最关键的算法——用户输入的是**拼音字母序列**，需要逐字母匹配当前汉字的拼音：

代码块

```
1   // lib/engine/PinyinStateMachine.ts
2
3   interface MatchState {
4     currentCharIndex: number;   // 当前匹配到第几个字符
5     currentPinyinPos: number;   // 当前拼音中匹配到第几个字母
6     inputBuffer: string;        // 当前累积输入
7     status: 'idle' | 'matching' | 'completed' | 'error';
8   }
9
10  class PinyinStateMachine {
11    private state: MatchState;
12    private poemData: PoemPinyinData;
13
14    constructor(poemData: PoemPinyinData) {
15      this.poemData = poemData;
16      this.state = {
17        currentCharIndex: 0,
18        currentPinyinPos: 0,
19        inputBuffer: '',
20        status: 'idle',
21      };
22    }
23
24    /**
25     * 处理单个按键输入
26     * @returns 匹配结果
27     */
28    processKey(key: string): MatchResult {
29      const currentChar = this.getCurrentChar();
```

```
    if (!currentChar) {
      return { type: 'completed', char: null };
    }

    // 标点处理：直接匹配标点字符
    if (currentChar.isPunctuation) {
      return this.handlePunctuation(key, currentChar);
    }

    // 汉字处理：匹配拼音序列
    return this.handlePinyinInput(key, currentChar);
  }

  private handlePinyinInput(
    key: string,
    currentChar: PinyinChar
  ): MatchResult {
    const expectedPinyin = currentChar.pinyinRaw; // "jiang"
    const expectedChar = expectedPinyin[this.state.currentPinyinPos];

    if (key.toLowerCase() === expectedChar) {
      // ✅ 正确输入
      this.state.currentPinyinPos++;
      this.state.inputBuffer += key;

      if (this.state.currentPinyinPos >= expectedPinyin.length) {
        // 当前汉字的拼音全部输入完成，推进到下一个字符
        const completedChar = currentChar;
        this.advanceToNextChar();
        return {
          type: 'char_complete',
          char: completedChar,
          correct: true,
        };
      }

      return {
        type: 'partial_match',
        char: currentChar,
        matchedCount: this.state.currentPinyinPos,
        totalCount: expectedPinyin.length,
        correct: true,
      };
    } else {
      // ❌ 错误输入
      return {
        type: 'error',
```

```
77          char: currentChar,
78          expected: expectedChar,
79          actual: key,
80          correct: false,
81        };
82      }
83    }
84
85    private handlePunctuation(
86      key: string,
87      currentChar: PinyinChar
88    ): MatchResult {
89      // 中文标点映射：允许输入对应的英文标点
90      const punctuationMap: Record<string, string[]> = {
91        ',': [','],
92        '。': ['.'],
93        '！': ['!'],
94        '？': ['?'],
95        '；': [';'],
96        '：': [':'],
97        '、': [',', '\\'],
98        '"': ['"'],
99        '"': ['"'],
100       '''': ["'"],
101       '''': ["'"],
102     };
103
104     const acceptedKeys = punctuationMap[currentChar.char] ||
    [currentChar.char];
105
106     if (acceptedKeys.includes(key)) {
107       this.advanceToNextChar();
108       return { type: 'char_complete', char: currentChar, correct: true };
109     }
110
111     return { type: 'error', char: currentChar, expected: currentChar.char,
    actual: key, correct: false };
112   }
113
114   private advanceToNextChar() {
115     this.state.currentCharIndex++;
116     this.state.currentPinyinPos = 0;
117     this.state.inputBuffer = '';
118
119     if (this.state.currentCharIndex >= this.getTotalChars()) {
120       this.state.status = 'completed';
121     }
```

```
  }

  getCurrentChar(): PinyinChar | null {
    // 遍历所有行，找到当前位置对应的字符
    let count = 0;
    for (const line of this.poemData.lines) {
      for (const char of line.chars) {
        if (count === this.state.currentCharIndex) {
          return char;
        }
        count++;
      }
    }
    return null;
  }
}
```

4.3 键盘事件处理（IME 兼容）

这是中文打字应用最坑的地方——必须正确处理 **IME 输入法事件**：

代码块

```
// lib/engine/KeyInputHandler.ts

/**
 * 核心：这里我们不依赖系统IME
 * 用户直接在英文键盘模式下输入拼音字母
 * 不触发系统输入法候选框
 *
 * 关键事件处理：
 * 1. 拦截 keydown/keyup，阻止默认行为
 * 2. 使用 compositionstart/compositionend 检测 IME 状态
 * 3. 只接受 a-z 字母和标点符号的直接输入
 */

class KeyInputHandler {
  private isComposing = false; // IME是否激活
  private engine: PinyinStateMachine;
  private stats: StatsTracker;

  constructor(engine: PinyinStateMachine, stats: StatsTracker) {
    this.engine = engine;
    this.stats = stats;
  }

  attach(element: HTMLElement) {
```

```
25      // 监听 composition 事件（检测IME状态）
26      element.addEventListener('compositionstart', () => {
27        this.isComposing = true;
28      });
29
30      element.addEventListener('compositionend', () => {
31        this.isComposing = false;
32      });
33
34      // 核心键盘事件
35      element.addEventListener('keydown', (e: KeyboardEvent) => {
36        // IME 激活时忽略（keyCode === 229 是 IME 标志）
37        if (this.isComposing || e.keyCode === 229) {
38          e.preventDefault();
39          return;
40        }
41
42        // 处理特殊键
43        if (e.key === 'Backspace') {
44          e.preventDefault();
45          this.handleBackspace();
46          return;
47        }
48
49        if (e.key === 'Enter') {
50          e.preventDefault();
51          this.handleEnter();
52          return;
53        }
54
55        if (e.key === 'Escape') {
56          e.preventDefault();
57          this.handleReset();
58          return;
59        }
60
61        // 只接受可打印字符
62        if (e.key.length === 1 && !e.ctrlKey && !e.metaKey && !e.altKey) {
63          e.preventDefault();
64          const timestamp = performance.now();
65          const result = this.engine.processKey(e.key);
66          this.stats.recordKeystroke(e.key, result, timestamp);
67          this.emitUpdate(result);
68        }
69      });
70    }
71  }
```

4.4 实时统计追踪器

代码块

```
// lib/engine/StatsTracker.ts

interface TypingStats {
  totalKeystrokes: number;   // 总按键数
  correctKeystrokes: number; // 正确按键数
  errorKeystrokes: number;   // 错误按键数
  totalCharsTyped: number;   // 已完成汉字数
  accuracy: number;          // 准确率 (%)
  kpm: number;               // 按键速度 (keys per minute)
  wpm: number;               // 打字速度 (chars per minute)
  correctWpm: number;        // 正确打字速度
  progress: number;          // 进度 (%)
  elapsedTime: number;       // 已用时间 (ms)
}

class StatsTracker {
  private startTime: number = 0;
  private keystrokes: KeystrokeRecord[] = [];
  private totalChars: number;
  private completedChars: number = 0;
  private errors: number = 0;

  constructor(totalChars: number) {
    this.totalChars = totalChars;
  }

  recordKeystroke(key: string, result: MatchResult, timestamp: number) {
    if (this.startTime === 0) {
      this.startTime = timestamp;
    }

    this.keystrokes.push({
      key,
      expected: result.char?.pinyinRaw?.[0] || '',
      correct: result.correct,
      timestamp,
    });

    if (!result.correct) {
      this.errors++;
    }

```

```
43      if (result.type === 'char_complete' && result.correct) {
44        this.completedChars++;
45      }
46    }
47
48    getStats(): TypingStats {
49      const now = performance.now();
50      const elapsed = now - this.startTime;
51      const elapsedMinutes = elapsed / 60000;
52
53      const total = this.keystrokes.length;
54      const correct = total - this.errors;
55
56      return {
57        totalKeystrokes: total,
58        correctKeystrokes: correct,
59        errorKeystrokes: this.errors,
60        totalCharsTyped: this.completedChars,
61        accuracy: total > 0 ? (correct / total) * 100 : 100,
62        kpm: elapsedMinutes > 0 ? total / elapsedMinutes : 0,
63        wpm: elapsedMinutes > 0
64          ? this.completedChars / elapsedMinutes : 0,
65        correctWpm: elapsedMinutes > 0
66          ? this.completedChars / elapsedMinutes : 0,
67        progress: (this.completedChars / this.totalChars) * 100,
68        elapsedTime: elapsed,
69      };
70    }
71  }
```

五、前端 UI 实现方案

5.1 项目目录结构

代码块

```
1  poetry-typing/
2  ├── prisma/
3  │   └── schema.prisma
4  ├── scripts/
5  │   ├── importPoems.ts          # 诗词数据导入
6  │   └── generatePinyin.ts       # 拼音预处理
7  ├── src/
8  │   ├── app/                    # Next.js App Router
```

```
│   │   ├── layout.tsx          # 根布局（暗色主题）
│   │   ├── page.tsx            # 首页
│   │   ├── course/
│   │   │   ├── page.tsx        # 课程浏览页
│   │   │   └── [category]/
│   │   │       └── page.tsx    # 分类课程列表
│   │   ├── practice/
│   │   │   └── [poemId]/
│   │   │       └── page.tsx    # 打字练习页
│   │   ├── stats/
│   │   │   └── page.tsx        # 统计/排行榜
│   │   └── api/                # API Routes
│   │       ├── auth/
│   │       ├── poems/
│   │       ├── practice/
│   │       └── stats/
│   ├── components/
│   │   ├── course/
│   │   │   ├── CourseGrid.tsx       # 课程网格
│   │   │   ├── PoemCard.tsx         # 诗歌卡片
│   │   │   ├── StarRating.tsx       # 星级评分
│   │   │   └── ProgressBar.tsx      # 底部进度条
│   │   ├── practice/
│   │   │   ├── TypingArea.tsx       # 打字主区域
│   │   │   ├── PinyinDisplay.tsx    # 拼音-汉字对齐显示
│   │   │   ├── PinyinChar.tsx       # 单个拼音字符组件
│   │   │   ├── Toolbar.tsx          # 顶部工具栏
│   │   │   ├── StatsBar.tsx         # 底部统计栏
│   │   │   ├── Cursor.tsx           # 打字光标
│   │   │   └── VirtualKeyboard.tsx  # 虚拟键盘(可选)
│   │   └── shared/
│   │       ├── ThemeToggle.tsx
│   │       └── LoadingSpinner.tsx
│   ├── lib/
│   │   ├── engine/                  # 打字引擎核心
│   │   │   ├── PinyinStateMachine.ts
│   │   │   ├── KeyInputHandler.ts
│   │   │   ├── StatsTracker.ts
│   │   │   └── index.ts
│   │   ├── pinyin/
│   │   │   ├── converter.ts         # 拼音转换工具
│   │   │   └── matcher.ts           # 拼音匹配算法
│   │   ├── db.ts                    # Prisma client
│   │   └── utils.ts
│   ├── hooks/
│   │   ├── useTypingEngine.ts       # 打字引擎 Hook
│   │   ├── useKeyboard.ts           # 键盘事件 Hook
```

```
56  │   │   └── useStats.ts          # 统计数据 Hook
57  │   ├── stores/
58  │   │   └── typingStore.ts          # Zustand 状态管理
59  │   └── types/
60  │       ├── pinyin.ts
61  │       ├── poem.ts
62  │       └── stats.ts
63  ├── public/
64  │   └── audio/                      # 朗读音频(可选)
65  ├── package.json
66  ├── next.config.ts
67  ├── tailwind.config.ts
68  └── tsconfig.json
```

5.2 拼音-汉字对齐渲染组件（核心UI）

这是 Type.fun 截图中最精髓的部分——**拼音和汉字的严格对齐排版**：

代码块

```
1   // components/practice/PinyinDisplay.tsx
2   'use client';
3
4   import { PinyinLine, PinyinChar } from '@/types/pinyin';
5
6   interface PinyinDisplayProps {
7     lines: PinyinLine[];
8     currentIndex: number;       // 当前打到的字符索引
9     currentPinyinPos: number;   // 当前拼音中的位置
10    errorIndices: Set<number>; // 错误的字符索引集合
11    showPinyin: boolean;        // 是否显示拼音
12  }
13
14  export function PinyinDisplay({
15    lines,
16    currentIndex,
17    currentPinyinPos,
18    errorIndices,
19    showPinyin,
20  }: PinyinDisplayProps) {
21    let globalIndex = 0;
22
23    return (
24      <div className="flex flex-col gap-8 font-mono">
25        {lines.map((line, lineIdx) => (
26          <div key={lineIdx} className="flex flex-col gap-1">
27            {/* 拼音行 */}
```

```
28            {showPinyin && (
29              <div className="flex flex-wrap gap-x-4 text-lg tracking-wider">
30                {line.chars.map((char, charIdx) => {
31                  const idx = globalIndex + charIdx;
32                  return (
33                    <PinyinUnit
34                      key={charIdx}
35                      char={char}
36                      globalIndex={idx}
37                      currentIndex={currentIndex}
38                      currentPinyinPos={currentPinyinPos}
39                      isError={errorIndices.has(idx)}
40                    />
41                  );
42                })}
43                {/* 行尾回车符 */}
44                <span className="text-gray-600 text-sm self-end">↵</span>
45              </div>
46            )}
47
48            {/* 汉字行 */}
49            <div className="flex flex-wrap gap-x-4 text-2xl tracking-widest">
50              {line.chars.map((char, charIdx) => {
51                const idx = globalIndex + charIdx;
52                const status = getCharStatus(idx, currentIndex, errorIndices);
53                return (
54                  <span
55                    key={charIdx}
56                    className={cn(
57                      'transition-colors duration-150',
58                      status === 'completed' && 'text-green-400',
59                      status === 'error' && 'text-red-400',
60                      status === 'current' && 'text-white',
61                      status === 'pending' && 'text-gray-500',
62                    )}
63                  >
64                    {char.char}
65                  </span>
66                );
67              })}
68            </div>
69
70            {(() => { globalIndex += line.chars.length; return null; })()}
71          </div>
72        ))}
73      </div>
74    );
```

```
75  }
76
77  // 单个拼音单元：逐字母着色
78  function PinyinUnit({
79    char,
80    globalIndex,
81    currentIndex,
82    currentPinyinPos,
83    isError,
84  }: {
85    char: PinyinChar;
86    globalIndex: number;
87    currentIndex: number;
88    currentPinyinPos: number;
89    isError: boolean;
90  }) {
91    if (!char.isChar) {
92      // 标点直接渲染
93      return <span className="text-gray-500">{char.char}</span>;
94    }
95
96    const pinyinLetters = char.pinyin.split('');
97    const isCurrent = globalIndex === currentIndex;
98    const isCompleted = globalIndex < currentIndex;
99
100   return (
101     <span className="inline-flex">
102       {pinyinLetters.map((letter, i) => {
103         let color = 'text-gray-500'; // 默认灰色
104
105         if (isCompleted) {
106           color = isError ? 'text-red-400' : 'text-green-400';
107         } else if (isCurrent) {
108           if (i < currentPinyinPos) {
109             color = 'text-green-400'; // 已输入正确
110           } else if (i === currentPinyinPos) {
111             color = 'text-white'; // 当前待输入（带光标）
112           }
113         }
114
115         return (
116           <span
117             key={i}
118             className={cn(color, 'transition-colors duration-100')}
119           >
120             {letter}
121             {/* 当前输入位置的下划线光标 */}
```

```
122                {isCurrent && i === currentPinyinPos && (
123                  <span className="absolute bottom-0 left-0 w-full h-0.5
124                                  bg-pink-500 animate-pulse" />
125                )}
126              </span>
127            );
128          })}
129        </span>
130      );
131    }
```

5.3 课程浏览网格组件

代码块

```
1   // components/course/CourseGrid.tsx
2
3   export function CourseGrid({ poems, userProgress }: CourseGridProps) {
4     return (
5       <div className="grid grid-cols-2 md:grid-cols-4 lg:grid-cols-7 gap-4 p-6">
6         {poems.map((poem) => {
7           const progress = userProgress?.[poem.id];
8           const isUnlocked = progress?.unlocked ?? false;
9           const stars = progress?.bestStars ?? 0;
10
11          return (
12            <PoemCard
13              key={poem.id}
14              poem={poem}
15              stars={stars}
16              isUnlocked={isUnlocked}
17              icon={getIconForMode(poem)}
18            />
19          );
20        })}
21      </div>
22    );
23  }
24
25  function PoemCard({ poem, stars, isUnlocked, icon }: PoemCardProps) {
26    return (
27      <Link
28        href={isUnlocked ? `/practice/${poem.id}` : '#'}
29        className={cn(
30          'relative flex flex-col items-center p-4 rounded-xl',
31          'bg-white/5 backdrop-blur border border-white/10',
```

```
        'transition-all duration-200',
        isUnlocked
          ? 'hover:bg-white/10 hover:scale-105 cursor-pointer'
          : 'opacity-50 cursor-not-allowed',
      )}
    >
      {/* 星级评分 */}
      {stars > 0 && (
        <div className="absolute top-2 left-2 flex gap-0.5">
          {Array.from({ length: 5 }).map((_, i) => (
            <Star
              key={i}
              size={12}
              className={i < stars ? 'fill-green-400 text-green-400' : 'text-gray-600'}
            />
          ))}
        </div>
      )}

      {/* 锁图标 */}
      {!isUnlocked && (
        <Lock size={16} className="absolute top-2 right-2 text-gray-400" />
      )}

      {/* 模式图标 */}
      <div className="w-12 h-12 flex items-center justify-center mb-3">
        {icon}
      </div>

      {/* 诗名和作者 */}
      <p className="text-xs text-center text-gray-300 leading-tight">
        《{poem.title}》{poem.dynasty}·{poem.author}
      </p>
    </Link>
  );
}
```

六、后端 API 设计

6.1 Next.js Server Actions（推荐方式）

代码块

```typescript
// app/actions/poems.ts
'use server';

import { prisma } from '@/lib/db';
import { cache } from 'react';

// 获取分类下的诗歌列表（带缓存）
export const getPoems = cache(async (category: string, page = 1) => {
  return prisma.poem.findMany({
    where: { category: { slug: category } },
    orderBy: { sortOrder: 'asc' },
    skip: (page - 1) * 50,
    take: 50,
    select: {
      id: true,
      title: true,
      author: true,
      dynasty: true,
      charCount: true,
      difficulty: true,
      tags: true,
    },
  });
});

// 获取诗歌详情（含拼音数据）
export const getPoemWithPinyin = cache(async (poemId: string) => {
  return prisma.poem.findUnique({
    where: { id: poemId },
    select: {
      id: true,
      title: true,
      author: true,
      dynasty: true,
      content: true,
      paragraphs: true,
      pinyinData: true,
      charCount: true,
    },
  });
});

// 保存练习记录
export async function savePracticeRecord(data: {
  poemId: string;
  accuracy: number;
  wpm: number;
```

```
48      kpm: number;
49      correctWpm: number;
50      duration: number;
51      charCount: number;
52      errorCount: number;
53      keystrokeLog?: any;
54    }) {
55      const session = await getServerSession();
56      if (!session?.user) throw new Error('Unauthorized');
57
58      const stars = calculateStars(data.accuracy, data.wpm);
59
60      // 事务：同时更新练习记录和课程进度
61      return prisma.$transaction(async (tx) => {
62        // 创建练习记录
63        const record = await tx.practiceRecord.create({
64          data: {
65            userId: session.user.id,
66            poemId: data.poemId,
67            accuracy: data.accuracy,
68            wpm: data.wpm,
69            kpm: data.kpm,
70            correctWpm: data.correctWpm,
71            duration: data.duration,
72            charCount: data.charCount,
73            errorCount: data.errorCount,
74            stars,
75            keystrokeLog: data.keystrokeLog,
76          },
77        });
78
79        // 更新课程进度
80        await tx.courseProgress.upsert({
81          where: {
82            userId_poemId: {
83              userId: session.user.id,
84              poemId: data.poemId,
85            },
86          },
87          create: {
88            userId: session.user.id,
89            poemId: data.poemId,
90            unlocked: true,
91            bestStars: stars,
92            bestAccuracy: data.accuracy,
93            bestWpm: data.wpm,
94            attempts: 1,
```

```
      },
      update: {
        bestStars: { set: Math.max(stars) },  // 取最高
        bestAccuracy: { set: Math.max(data.accuracy) },
        bestWpm: { set: Math.max(data.wpm) },
        attempts: { increment: 1 },
      },
    });

    // 解锁下一首诗
    if (stars >= 3) {
      await unlockNextPoem(tx, session.user.id, data.poemId);
    }

    return record;
  });
}

// 星级计算逻辑
function calculateStars(accuracy: number, wpm: number): number {
  if (accuracy >= 98 && wpm >= 40) return 5;
  if (accuracy >= 95 && wpm >= 30) return 4;
  if (accuracy >= 90 && wpm >= 20) return 3;
  if (accuracy >= 80 && wpm >= 15) return 2;
  if (accuracy >= 60) return 1;
  return 0;
}
```

6.2 REST API Routes（备选/兼容方式）

代码块

```
// app/api/poems/[category]/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { prisma } from '@/lib/db';

export async function GET(
  request: NextRequest,
  { params }: { params: { category: string } }
) {
  const { searchParams } = new URL(request.url);
  const page = parseInt(searchParams.get('page') || '1');
  const limit = parseInt(searchParams.get('limit') || '50');

  const poems = await prisma.poem.findMany({
    where: { category: { slug: params.category } },
```

```
    orderBy: { sortOrder: 'asc' },
    skip: (page - 1) * limit,
    take: limit,
  });

  return NextResponse.json({
    data: poems,
    pagination: { page, limit, total: await prisma.poem.count() },
  });
}
```

七、状态管理方案

使用 **Zustand**（轻量、TypeScript 友好、性能好）：

代码块

```
// stores/typingStore.ts
import { create } from 'zustand';

interface TypingState {
  // 诗歌数据
  poemData: PoemPinyinData | null;

  // 引擎状态
  currentCharIndex: number;
  currentPinyinPos: number;
  completedChars: Set<number>;
  errorChars: Set<number>;

  // 统计
  stats: TypingStats;

  // 设置
  showPinyin: boolean;
  enableReadAloud: boolean;
  enableFingerGuide: boolean;

  // Actions
  initPoem: (data: PoemPinyinData) => void;
  processKey: (key: string) => MatchResult;
  reset: () => void;
  togglePinyin: () => void;
  toggleReadAloud: () => void;
  updateStats: () => void;
```

```
29 }
30
31 export const useTypingStore = create<TypingState>((set, get) => ({
32   poemData: null,
33   currentCharIndex: 0,
34   currentPinyinPos: 0,
35   completedChars: new Set(),
36   errorChars: new Set(),
37   stats: initialStats,
38   showPinyin: true,
39   enableReadAloud: true,
40   enableFingerGuide: false,
41
42   initPoem: (data) => {
43     set({
44       poemData: data,
45       currentCharIndex: 0,
46       currentPinyinPos: 0,
47       completedChars: new Set(),
48       errorChars: new Set(),
49       stats: initialStats,
50     });
51   },
52
53   processKey: (key) => {
54     const state = get();
55     // ... 状态机处理逻辑（调用 PinyinStateMachine）
56     // 返回 MatchResult 并更新状态
57   },
58
59   reset: () => {
60     set({
61       currentCharIndex: 0,
62       currentPinyinPos: 0,
63       completedChars: new Set(),
64       errorChars: new Set(),
65       stats: initialStats,
66     });
67   },
68
69   togglePinyin: () => set((s) => ({ showPinyin: !s.showPinyin })),
70   toggleReadAloud: () => set((s) => ({ enableReadAloud: !s.enableReadAloud })),
71 }));
```

八、TTS 语音朗读方案

Type.fun 支持"跟读"功能，实现方案有两条路：

方案 A：Web Speech API（免费、零成本）

代码块

```
// lib/tts/webSpeechTTS.ts

export function readAloudPoem(text: string) {
  if (!('speechSynthesis' in window)) {
    console.warn('Web Speech API 不支持');
    return;
  }

  const utterance = new SpeechSynthesisUtterance(text);
  utterance.lang = 'zh-CN';
  utterance.rate = 0.8; // 慢速朗读
  utterance.pitch = 1.0;

  // 选择中文语音
  const voices = speechSynthesis.getVoices();
  const zhVoice = voices.find(v => v.lang.startsWith('zh'));
  if (zhVoice) utterance.voice = zhVoice;

  speechSynthesis.speak(utterance);
}

// 逐句跟读（跟着用户打字进度朗读）
export function readAloudLine(
  line: string,
  onEnd?: () => void
) {
  const utterance = new SpeechSynthesisUtterance(line);
  utterance.lang = 'zh-CN';
  utterance.rate = 0.7;
  utterance.onend = onEnd ?? null;
  speechSynthesis.speak(utterance);
}
```

方案 B：第三方 TTS API（高质量）

代码块

```
// 阿里云 / 百度 TTS API
```

```
// 预先生成音频文件，按诗歌 ID 存储
// /public/audio/{poemId}.mp3

export async function getAudioUrl(poemId: string): Promise<string> {
  // 先检查是否已有预生成音频
  const cached = `/audio/${poemId}.mp3`;
  if (await audioExists(cached)) return cached;

  // 调用 TTS API 生成
  const response = await fetch('/api/tts/generate', {
    method: 'POST',
    body: JSON.stringify({ poemId }),
  });
  return response.json().then(r => r.audioUrl);
}
```

九、性能优化策略

9.1 渲染优化

代码块

```
// 1. 拼音组件使用 React.memo 避免不必要重渲染
const PinyinChar = React.memo(function PinyinChar({ ... }) {
  // 只有 status 变化时才重新渲染
});

// 2. 使用 requestAnimationFrame 节流统计更新
function useThrottledStats() {
  const rafRef = useRef<number>();

  const updateStats = useCallback(() => {
    if (rafRef.current) cancelAnimationFrame(rafRef.current);
    rafRef.current = requestAnimationFrame(() => {
      // 更新统计 UI
    });
  }, []);
}

// 3. 虚拟滚动（长诗才需要）
// 对于唐诗三百首大部分篇幅不长，暂不需要
```

9.2 数据加载优化

代码块

```
// Next.js ISR 增量静态生成（课程列表页）
export const revalidate = 3600; // 1小时缓存

// 诗歌详情页 SSG 预生成
export async function generateStaticParams() {
  const poems = await prisma.poem.findMany({
    select: { id: true },
    where: { category: { slug: 'tang-poetry' } },
  });
  return poems.map((p) => ({ poemId: p.id }));
}
```

十、部署方案

10.1 方案 A：Vercel 部署（推荐快速上线）

代码块

```
# 1. 安装 Vercel CLI
pnpm add -g vercel

# 2. 配置环境变量
vercel env add DATABASE_URL
vercel env add NEXTAUTH_SECRET

# 3. 部署
vercel --prod
```

优势：Next.js 原生支持、CDN、Serverless Functions 自动伸缩。

10.2 方案 B：Docker 自建（完全掌控）

代码块

```
# Dockerfile
FROM node:22-alpine AS base
RUN corepack enable pnpm

FROM base AS builder
WORKDIR /app
COPY pnpm-lock.yaml package.json ./
```

```
RUN pnpm install --frozen-lockfile
COPY . .
RUN pnpm prisma generate
RUN pnpm build

FROM base AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./.next/static
COPY --from=builder /app/public ./public

EXPOSE 3000
CMD ["node", "server.js"]
```

代码块

```
# docker-compose.yml
version: '3.8'
services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/poetry
      - REDIS_URL=redis://redis:6379
    depends_on:
      - db
      - redis

  db:
    image: postgres:17-alpine
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      POSTGRES_DB: poetry
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass

  redis:
    image: redis:7-alpine

volumes:
  pgdata:
```

十一、开发计划与里程碑

阶段	周期	交付物	说明
Phase 1: 基础骨架	1周	项目初始化、数据导入、基础UI	Next.js 项目搭建、Prisma schema、唐诗数据导入、拼音预处理
Phase 2: 打字引擎	2周	核心打字功能可用	PinyinStateMachine、KeyInputHandler、拼音-汉字对齐渲染、颜色反馈
Phase 3: 统计系统	1周	实时统计 + 评分	StatsTracker、星级评分、底部统计栏、练习记录保存
Phase 4: 用户系统	1周	登录/注册、进度保存	NextAuth.js 集成、用户进度同步、课程解锁机制
Phase 5: 课程浏览	1周	完整课程浏览体验	网格卡片、分类筛选、搜索、解锁状态展示
Phase 6: 打磨优化	1周	生产就绪	性能优化、TTS 集成、响应式适配、错误处理、部署上线

总计：约 7 周（一个人 full-time 的节奏）

十二、关键技术风险与应对

风险	概率	影响	应对
IME 输入法冲突	高	体验灾难	**强制英文模式提示** + 拦截 composition 事件 + keyCode 229 检测
多音字拼音歧义	中	匹配错误	pinyin-pro 自带上下文多音字处理、导入时人工校验高频诗词
拼音对齐排版困难	中	UI 错位	使用等宽字体 + CSS Grid/Flexbox + 以拼音最长字符为基准宽度
移动端键盘兼容	中	移动端不可用	Phase 1 先做 PC 版，后续可考虑虚拟键盘方案
大量诗词加载性能	低	首屏慢	ISR/SSG 预生成 + 分页 + 数据按需加载

IME 处理的关键策略

说白了，这个应用的核心用户体验是——**用户需要切到英文键盘模式，直接敲拼音字母**，而不是通过系统输入法组合汉字。这一点 Type.fun 做得很好，它本质上是在做一个"**拼音打字跟打器**"而不是"汉字输入法"。

所以：

1. **进入练习页时，弹出提示**：请切换到英文输入模式
2. **检测到 `compositionstart` 时**，显示警告 Toast：检测到中文输入法，请切换到英文模式
3. **keyCode === 229**（IME 键码）时，忽略输入并提示

十三、总结：为什么这个方案可行

1. **纯 Node.js 全栈**：Next.js 15 一把梭，前后端同构，没有语言切换的心智负担
2. **核心引擎纯客户端**：打字匹配完全在浏览器运行，零延迟、离线可用
3. **数据源成熟**：chinese-poetry 项目已有 49K Star，5.5万首唐诗，拿来就用
4. **拼音转换可靠**：pinyin-pro 是 JS 生态最强的拼音库，支持多音字、声调、声母韵母
5. **架构清晰**：数据层 Prisma、状态层 Zustand、UI 层 React Components，边界分明

6. **部署灵活**：Vercel 一键部署，或 Docker 自建，随你选

杨晨 2026年03月24日 17:01